



Programming Fun(damentals): Using commercial video games to teach basic coding to adult learners

Joan Arnedo-Moreno^{*}, David García-Solórzano

Universitat Oberta de Catalunya, Rambla de Poblenou, 156, Barcelona, 08018, Spain

ARTICLE INFO

Keywords:

e-learning
Game based learning
Video games
Programming games
Computational thinking
Self-efficacy

ABSTRACT

The acquisition of computational thinking and coding skills is of special significance in helping adult learners keep pace with a new context where those skills are required for many parts of the workforce. However, there is an agreement in the scientific literature that important challenges exist, such as keeping student engagement and the requirement of developing problem-solving skills, and not just learning the code syntax. The inclusion of digital Game-Based Learning (GBL) has shown great potential. To better understand how it can be helpful to adult learners in STEM degrees, a study was conducted on the application of two very popular commercial video games in different introductory programming courses at an online university, during the introduction of basic programming topics (CS1). The methods combined a descriptive qualitative and a quantitative approach, using a reflective journals and questionnaires, which helped students consider and express their experiences, how they interacted with the games, how the games helped them better understand the topics, and realize their personal progress and potential. Results showed that the use of these games as educational resources had a positive impact in their engagement and sense of self-efficacy, but only if some important instructional considerations are taken into account.

1. Introduction

Programming skills have gained special relevance in many areas of education, beyond the classical Computer Science degrees. One of the key reasons is the return to prominence of the concept of computational thinking [1,2], which is now contemplated on equal terms with other 21st century skills, such as creativity or critical thinking [3]. Initiatives to promote the acquisition of this skillset are considered beneficial to the citizenship at large [4,5] and, as a result, an emphasis has been put in incorporating programming into school curricula. It is now commonplace in many institutions at all stages of education to use resources specifically aimed at kids, such as LOGO-like applications and robotics, or creative development environments, such as Scratch (more than a decade old at the time of this writing) [6,7]. However, some consideration should also be given to adults who now live in a world where programming has become an instrumental skill across different scopes, intertwined with hot topics that have a big impact in the workplace, such as internet of things, e-health, big data, artificial intelligence or deep learning, among others.

According to the scientific literature, learning computer programming faces some important challenges [8,9]. Scholars on the topic agree that the most important one is the fact that students tend to focus on the low level technical aspects, such as language syntax, instead

of first developing their capability of abstraction, and problem-solving skills [10,11]. Another important difficulty found at the top is usually student engagement. Therefore, there is a well-established continuous exchange of ideas between researchers about how to reduce the friction when presenting introductory programming to novice students [12,13]. In this search for new methods to facilitate the entry to computer programming, there is an obvious emphasis in introducing new digital tools, following the general trends in educational tools [14,15]. Given the aforesaid challenges detected in the literature, these tools should not be limited to the coding stage (e.g. powerful IDEs), but also help learners acquire the required abstraction skills, and keep their interest.

Bearing in mind the challenges of learning programming, and regardless of the target population, young or adult, it may make sense to take inspiration from digital Game-Based Learning (GBL) [16] approaches. Following the process of digital transformation in education, computer video games have gained traction as a method to instruct a wide breadth of topics, especially due to their ability to enhance motivation, as well as learners' fundamental skills such as abstract problem-solving and strategic thinking, according to the scientific literature [17,18]. In this regard, a video game genre colloquially known as *programming games* has raised in popularity in recent years, even among mainstream players. Such games require players to use elements

^{*} Corresponding author.

E-mail address: jarnedo@uoc.edu (J. Arnedo-Moreno).

of computer programming, such as writing a script, in order to interact with them.

However, a big hurdle in using programming games as an educational resource is the cost of developing them with good production values, which is not trivial. Therefore, it may be worth to first study existing commercial off-the-shelf (COTS) games. It could be argued it goes against conventional wisdom to choose a mainstream commercial game, which was designed with entertainment as its main purpose, to teach a challenging concept such as programming. But it is worth hypothesizing that, by subjecting them to the same degree of scrutiny as any other educational tool, making sure that students found them helpful in reflecting on their skills and did not just learn how to beat them, some could be found actually useful [19,20]. Finding the appropriate games would also allow educators to benefit from their design principles, with a proven track record on engagement mechanics and usability, as shown by their ability to reach the conventional gaming population.

To this end, this paper presents a study that examined the impact of two well-known and highly rated COTS programming games, named “Human Resource Machine” (HRM) and “7 Billion Humans” (7BH) [21, 22], in a group of adult learners (average age 30+) across different technical bachelor degrees during their first semester learning computer programming. Even though these games have a colorful interface and a PEGI/ESRB¹ rating for all public, they are challenging, not specifically aimed at children, and certainly require the development of some degree of algorithmic skills to be able to advance. This study was performed in a fully online university, in the context of a project with the purpose of developing the curriculum for a programming school open to anybody who wanted to learn programming, regardless of their technical background. A mixed methodology was used to assess both games in terms of their influence during the learning process, their usability, and the change of perception in the students’ sense of self-efficacy, defined by Bandura [23] as “people’s beliefs about their capabilities to produce designated levels of performance that exercise influence over events that affect their lives”, determining how people feel, behave, and motivate themselves.

This paper is structured as follows. To provide a theoretical background, Section 2 examines both the key topics in a programming curriculum in higher education according to the academic literature and the programming genre of video games itself, in order to study how they can be reconciled. The main research questions are formalized in Section 3, which describes the methodological framework of the study, mainly its context of application, video game selection criteria and enrollment process. The results are presented in Section 4 and further discussed, according to the research questions, in Section 5. This paper is concluded in Section 6, summarizing the main findings of the study and proposing some guidelines for the application of programming games, as well as future work.

2. Background

This section presents the contextualization for the study. First, the main properties of the typical computer programming curriculum, such as its knowledge areas, are examined. Next, the genre of programming games and their current context are described, also providing a brief overview of the particular characteristics of HRM and 7BH. The final section aligns the two previous ones by reviewing how, according to the current scientific literature, video games can be used to teach programming, and which ones are found to be aligned to the knowledge areas expected in an introductory programming course.

2.1. The programming curriculum in higher education

Attempts to standardize the programming curriculum, and computer science education at large, have undergone successive iterations in the past decades, as far as back as the year 2001, through a joint effort by an ACM and IEEE Task force. As a result of this process, the 2013 Computer Science Curricula (or CS2013) [24] was published, becoming the most overwhelmingly popular approach in the scientific literature when studying how to develop computer science programs. The CS2013 is organized into a set of 18 Knowledge Areas (KAs), corresponding to topical areas of study in computing. Four of them can be considered directly related to any part of the software development process: *Software Development Fundamentals (SDF)*, *Algorithms & Complexity (AL)*, *Software Engineering (SE)* and *Programming Languages (PL)*. They encompass from the most abstract layer of problem definition, to the more tangible task of generating correct code using a specific programming language.

In this regard, the Software Development Fundamentals (SDF) area is considered the precondition to the rest, or quoting the report itself, “identifying those concepts and skills that should be mastered in the first year of a computer science program”. SDF is further divided and structured into four units (see Fig. 1), each almost equally balanced to the rest in number of expected hours assigned to the curriculum, thus providing an excellent starting point when designing any introductory programming course. A good amount of the abstract problem-solving concepts could be considered to be included in *Algorithms and Design*.

In order to adapt to the changing dynamics of computing education, the CS2013 report was greatly overhauled seven years later in the Computing Curricula 2020 (or CC2020) [25]. The main reason being that, at the present time, some of these skills no longer concern to Computer Science graduates only, and have become relevant to other fields with specialized educational programs of their own, such as Data Science or Cybersecurity. Likewise, the CC2020 report also moved to a focus on competencies, transitioning from knowledge-based to competency-based learning. In his regard, a competency was defined as a composition of Knowledge-Skill-Disposition (K-S-D) dimensions observed within the performance (T) of a task, rated using Bloom’s Taxonomy [26]. In the CC2020, computer programming is located in *Programming Fundamentals*. However, the information provided on this area is not very granular, in contrast with the previous one. For instance, the abstract problem-solving skill in computer programming is included under the single competency “Create an appropriate algorithm to illustrate iterative, recursive functions, as well as divide-and-conquer techniques and use a programming language to implement, test, and debug the algorithm for solving a simple industry problem”. Therefore, even when this report has been available for several years, the CS2013, with its division into clear subareas and subtopics, is still popular.

Apart from these very well-known frames of reference, there are a number of studies and reviews in the scientific literature, such as [27, 28] among others, which analyze the properties of introductory programming courses being offered around the world. These studies cover aspects like how such courses are structured, which topics are considered the most relevant ones, what methodological approaches are applied, and what general skills are deemed important to succeed. Examining them may serve as a more practical approach to discern the key parts of an effective introductory programming course. Usually, most differences detected among courses seem to fall on how to introduce the most abstract areas of knowledge, or the choice of the programming language. However, there are a few syllabus concepts that seem to be frequently explicitly stated, such as “Problem-solving”, “Debugging”, or the very specific “Selection statements”. Additionally, other non-technical skills linked to being able to become a good programmer encompass critical thinking, creativity, persistence and teamwork. Nevertheless, on a closer inspection, their conclusions are still mostly aligned with the topics proposed in the CS2013, and only partially, with the CC2020.

¹ <https://pegi.info>.

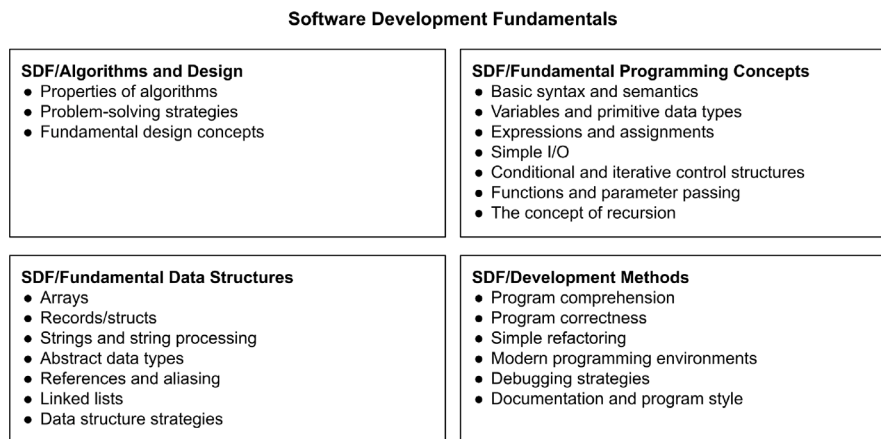


Fig. 1. Units and topics included in the Software Development Fundamentals knowledge area of the 2013 Computer Science Curricula (CS2013).

2.2. Programming games

The concept of *programming games* exists since 2001, at least, defined as games where the player “writes short programs that control agents within a game. These agents then compete and react to situations based on the player’s programming” [29]. Hence, programming games present challenges that require the player to develop an algorithm, and often to write code, in order to overcome them. One of the core features of most programming games is that the player cannot directly control a game avatar, relying instead on instructions chosen *a priori* through the interface, and then executed, just like a computer program. Additionally, it is also common that the user interface contains many computer-related visual metaphors (e.g. computer internals). How the programming of each avatar action is achieved, from a user standpoint, may vary from game to game, but the most common methods to create the programs follow:

- As a state graph, or graphical flow chart.
- As a sequence of fully graphical blocks, each acting as a metaphor of an action, usually movements in a LOGO-like environment.
- As a sequence of parametrizable blocks from a set of predefined textual instructions, i.e. a Scratch-like interaction.
- As written code, like in a real programming environment, using a real language, or a simpler made up one.

It is worth mentioning that some programming games also include features from other genres, such as adventure or platform, and the programming part is just an aspect of the whole experience (e.g. to solve specific puzzles).

2.2.1. Programming games in the gaming ecosystem

Despite having been formally identified and categorized for decades, programming games have always been a niche genre, primarily aimed at kids, relying on a LOGO-like approach and focusing on the movement of the avatar itself (e.g. reach a location while avoiding obstacles). However, in recent years, the interest on them seems to have grown significantly, and consequently, they have surfaced to the mainstream. Most are developed by indie studios with a clear goal in entertainment first and foremost, targeted to the kind of teen or adult players that would enjoy programming-like logical challenges, instead of defeating enemies or platform-jumping. The most popular ones can be found as commercial off-the-shelf (COTS) games in digital distribution platforms such as Steam² or “Good Old Games” (GOG),³ sold side by side with important AAA video game franchises.

Nevertheless, the educational edge is not lost by the developers, and some of them clearly have such purpose, or could even be considered straight Serious Games (SG) [30]. Indeed, a quick glance at Steam, the most popular video game digital distribution platform among PC users with around 120.4 million active monthly players, can provide an idea. According to Steam stats as of January 2024, there are 536 programming games on sale. To better gauge the significance of this number, this figure is similar to the number of political or transportation simulators (531 and 501 titles, respectively). It is also a bit below the number of MOBAs (574 titles), a very popular genre in the e-sports field (e.g. DOTA2, League of Legends). However, it understandably pales in comparison to very popular longstanding genres, such as arcade games (13 978 titles), and their sales and number of players cannot compete with big AAA franchises.

Fig. 2 shows some additional detail in two separate charts, with the number of new games published under this genre by year interval, and the preferred cross-genres (i.e. number of programming games that can also be considered simultaneously as part of that other genre), following the nomenclature in Wolf’s taxonomy [29]. In that regard, this trend shows that the act of programming has now become one among the many ways a game can become, as Hunnicke or Lazzaro state in their studies [31,32], “an obstacle course”, given that “humans must be challenged” to evoke desirable emotional responses, just like defeating enemies or jumping across chasms in other genres. Nevertheless, the “Education” tag is still certainly prevalent, although, interestingly, not at the very top, emphasizing that programming games are mainly considered engaging entertainment products at present time.

2.2.2. “Human Resource machine” (HRM) and “7 Billion Humans” (7BH)

“Human Resource machine” and “7 Billion Humans” (HRM and 7BH, from now on) are two examples of programming games, designed and developed as entertainment products by Tomorrow Corporation, and published in 2015 and 2018, respectively. They share the same storytelling (a tale of corporate hierarchy ascension by performing tasks assigned by the boss), look&feel, game mechanics and user interface. However, the former focuses on solving challenges by the application of iterative structures to control a single avatar, whereas in the latter the player may control several avatars at once and focuses on conditional structures and synchronization. Although they look simplistic at first glance, even child-like, they become quite challenging after a few levels, and the last levels do require some effort, even for a seasoned programmer.

Fig. 3 shows a sample screenshot from each game, representing a typical level. Both games use an eminently visual and colorful interface, represented as a single game screen with different areas. The program creation area is at the right, and the instructions are represented with colored blocks that are dragged and dropped. The player avatar represents a small office drone that obeys the instructions of this program,

² <https://store.steampowered.com>.

³ <https://www.gog.com>.

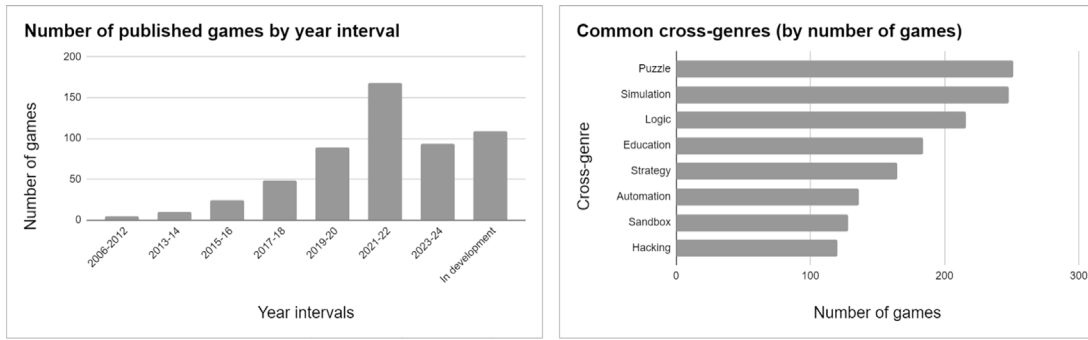
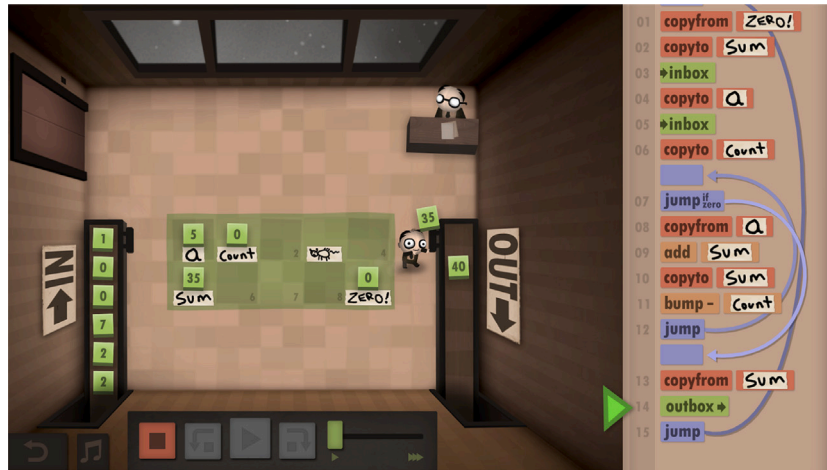


Fig. 2. Overview of the programming games published in Steam.



(a) Human Resource Machine



(b) 7 Billion Humans

Fig. 3. Sample screenshots from the games (HRM and 7BH). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

moving around the main central area. At the bottom, the program control buttons can be found: play, stop, execute a single step forward or backwards, and execution speed.

In HRM, the main screen area has two conveyor belts at opposite sides, processing input ($\rightarrow$ IN) and output (OUT $\rightarrow$) sequences of values. In the center, there is a special-tiled area, painted in green, where the avatar can store and manipulate values, behaving like the computer memory. The goal of the game is to follow the boss orders in order to produce a correct output sequence given an input sequence. For

instance, to multiply every input value by eight, or only to output values above 0, discarding the rest.

The instruction blocks can be simple arithmetic expressions and increment/decrement (add, sub, bump- or bump+), flow control statements (jump, jump if zero, jump if negative), or value transportation ones. In this case, the avatar needs not be directed, he automatically moves to the expected destination. He can be told to pick up a value from the input conveyor belt and hold it above his head, or to drop the value he is currently holding into the output one ($\rightarrow$ input, $\rightarrow$ output). It can also be told to make a copy of any value stored in

any tile green and hold it, discarding any other value he was holding up to that moment, or to store a copy of the value he is holding into a green tile, also overwriting any previous value (copyfrom, copyto). Furthermore, some instruction blocks also accept parameters in text form (see Zero! or Count in Fig. 3(a)). Correct solutions are graded by number of instructions and steps used. Moreover, programs can be executed step-by-step for debug and optimization (best score) purposes. A program automatically ends as soon as all input values have been correctly processed and dropped into the out conveyor belt.

Fig. 3(a) includes an example of code for level 20 (out of 41) in HRM, named “*Multiplication Workshop*”. The goal is described by the boss as “*For each two things in the inbox, multiply them, and outbox the result. Don’t worry about negative numbers for now. You got... LABELS! They can help you remember the purpose of each tile on the floor. Just tap any tile on the floor to edit*”. Explained in broad terms, a possible solution that does not try any optimization could be that the player initializes Sum to 0 (instructions 1 and 2), and reads two values from the input conveyor belt (instructions 3 and 5). The first one will be stored in Q, and the second one in Count (instructions 4 and 6). Sum will act as an accumulator, where Q is successively added Count times (instructions 7–12). The end result is dropped into the output conveyor belt (instructions 13 and 14). Finally, the process starts again until no more value pairs are available in the input conveyor belt (instruction 15). The game always guarantees that, in any level, the input value sequence format is correct and does not require checking. In this case, that its length is always even numbered.

As for 7BH, being a game published three years later, the problem design approach changed substantially, making it a more complex game. Even though the interface and its basic mechanics regarding program construction are very similar, there are no conveyor belts or special green tiles in the main area. Now the values that require manipulation can be spread across the screen, and this task is performed by coordinating the efforts of different avatars which act simultaneously, all executing exactly the same program. Therefore, conditional statements play a very important role, and some synchronization is required. Moreover, whereas HRM focuses on data storage and manipulation problems, 7BH stresses controlling the movement of the different avatars and the transportation of values to specific tiles at any location in the level, properly scanning the environment before acting (e.g. spotting holes where the avatar can fall).

Therefore, the instruction set has undergone important changes, and can be divided into physical, threading, memory and synchronization operations. Some of them have different options or kinds of parameters, so only a brief overview is provided here. On one hand, physical operations allow avatars to manipulate the environment. The avatars can still try to pick up and drop values at their position (pickup, drop), but now they can also be ordered to explicitly move a step in some direction (step). All the avatars can also exchange values they are holding between themselves (giveTo, takeFrom), and values can now be modified at will, without the need to copy from an existing one (write). On the other hand, threading operations control program flow and include jumping to any line of code (jump), conditionals (if), ending the execution of the program for an avatar (end), and performing some operations around all directions, one at a time, clockwise (forEachDir).

A very important difference in comparison with HRM is that, starting from level 25, the player has memory slots with different data types (integer, pointer-like references to screen positions, etc.), which can be freely accessed (set) or used to perform any of the four basic arithmetic operations (calc). They can also be used to search for the element of a specific data type that is the nearest one to an avatar (nearest). Finally, threading operations are also another innovation from HRM, allowing avatars to synchronize their work by sending messages, or waiting until they listen to them (tell, listen).

Fig. 3(b) includes an example of code for level 7 (out of 69) in 7BH, named “*Collation station*”. As shown, the boss asks the player to

“*Organize the data cubes into a nice line along the bottom row, between the two arrows*”. Therefore, the player has to be able to control all the seven avatars at once with a single program, making them walk down in a straight line, picking up the cube when they find it, taking into account that they are located in different rows for each avatar, and dropping the cube to the floor just before the hole. In this example of code, the same process keeps repeating (instruction 7). Avatars take a single step downwards and check for a hole in front of them (instructions 1 and 2). Finding a hole means they reached the end of the way, so they drop the cube at their position, as it is necessary to complete the boss instructions (instruction 3). Next, they check whether a cube is on the floor in front of them instead (instruction 4). If this is the case, they take a step and pick it up from their new position (instructions 5 and 6). It is worth noting that, in this program, the avatars will end up falling down the hole, but the only condition is to properly organize the data cubes, not necessarily to allow their survival. Disposing of the avatars is actually an optimization, since it is how this example guarantees the termination of each instance of the program as soon as each avatar completes their task.

Additional examples of code, including optimizations, and more detailed instruction set descriptions can be found in the Steam guides page of HRM⁴ and 7BH.⁵

2.3. Video games in a programming curriculum

The use of video games as educational tools to help develop computational thinking or basic programming skills has been studied in the literature, and it is possible to find different research works supporting its use in formal education. On further examination, the current state of the art can be summarized in several successive reviews [30,33,34] that provide a vast list of programming video games that have been used in class or show some potential, and how their game mechanics may align to some parts of an expected programming curriculum at different stages of education. Among all three, 159 different games are discussed, taking some repetitions between papers into account. This global list encompasses both bespoke games that were created with a clear educational purpose in mind –to be used in class and explicitly referred to as “serious games for learning programming”–, and games that just self-label as “programming games”, being more geared towards mainstream entertainment.

In order to study the alignment of each video game with the expected curriculum, the most popular approach seems to be the subtopics in the SDF knowledge area proposed in the CS2013 (see Section 2.1). Even though, in some cases, there are some changes in the use of the nomenclature by the authors, all reviews are very similar in their approach and end up including the most obvious topics expected in an introductory programming course. Table 1 presents a brief summary of the most popular learning outcomes in this kind of video games, according to each study. This list shows that most programming games include only very basic topics of an introductory programming curriculum, mostly flow control and variable operation. The numbers also allows gauging the expected difficulty in finding an appropriate programming game for a specific set of topics.

For instance, according to Table 1, the choice of games when looking for one that helps in learning “Input/Output” will be very limited, since very few take this topic into account, whereas a wide breadth of them include conditions and/or iterations. It is also interesting that, according to this summary, it can be concluded that some of the games in the reviews do not even qualify for any of the topics. On a closer look, at best, they are more like puzzle games, maybe useful to develop general problem-solving skills, but no actual programming-specific topics are considered. At worst, some claim to be part of the

⁴ <https://steamcommunity.com/app/375820/guides>.

⁵ <https://steamcommunity.com/app/792100/guides>.

Table 1
Sample of learning outcomes and number of games that include it, by study.

Learning outcome	Vahldick (40 games)	Miljanivoc (49 games)	Arnedo-Moreno (70 games)
Variables	3	23	24
Basic syntax	0	30	19
Conditions and/or iterations	1	30	29
Functions	10	18	10
Arrays	2	13	10
Input/Output	1	8	1

“programming” genre, but they do not actually require any actual computational thinking or programming skills, relying only on their aesthetics or graphical metaphors to make that claim. This is a very important aspect that must be taken into account when evaluating COTS programming games, widely available in digital distribution platforms, for an educational purpose.

An additional categorization used by some of the reviews that can also be useful is by its intended target audience. The largest audience for most of the games identified as “serious games for learning programming” are undergraduate students (18+ years old), followed by children (5–13 years old) and high school students (13–17 years old). It is worth noting that, even though undergraduate studies are formally considered ages above 18 years old, here it seems to mostly refer to a young population. Studies on adults or seniors, or students in a lifelong learning educational context in general, are never explicitly considered. Likewise, and given this categorization, the studies clearly show that most of the games geared towards a young pre-university audience are variations of LOGO-like games, where the player has to control the movement of an avatar to complete an obstacle course.

From these studies, it is possible to assess *a priori* which are the units and topics from the SDF knowledge area of the 2013 Computer Science Curricula (CS2013), as shown in Fig. 1, where video games can play a better role in helping students. The obvious choices would be in the development of skills related to “Problem solving-strategies” and “Properties of Algorithms”. In the more concrete aspects, many programming games let the player experiment with concepts very similar to “Variables and primitive data types”, “Expressions and assignments”, and especially “Conditional and iterative control structures”. Indirectly, players will also need to develop their “Program comprehension” and “Debugging strategies” to succeed when playing these games. The rest of the topics, usually, can only be estimated in a game-by-game basis. For instance, few games use a real programming language syntax or support topics from the “Fundamental Data Structures” area.

To sum up, there is currently a broad spectrum of programming games that could be considered as educational resources in a programming curriculum at different stages of education. There is ample evidence in the scientific literature that a sizeable amount of them seem to align to recommendations such as the ones from ACM/IEEE, thus fulfilling the basic tenets of an educational process [16,35] and confirming the general proposition that a well-designed video game can also contain sound learning principles [36]. However, one must be very careful, since not all games labeled as “programming” are useful for every topic, and in fact, for a sizeable amount, it would be better to just consider them as just puzzle games.

3. Methodology

This section describes, step by step, what this study intends to answer and why, its motivation and context, and all the details related to the methodological approach of the research.

3.1. Research objectives and questions

This study sets out to analyze the use of COTS video games as an educational resource to help to reduce the challenges a novice student, especially adult learners, must face in an introductory programming curriculum (CS1). In that regard, Medeiros et al. [12] identify which are the main difficulties for beginners in a very thorough literature review. According to these authors, the top five difficulties in the following order are (in parentheses, number of studies found reaching that conclusion, out of 100): engagement (36), problem-solving (23), control structures (12), confidence (10), and the abstract nature of programming (6). Seeing that the more prominent items in this list are engagement and problem-solving, it is worth hypothesizing that Games-Based Learning, using appropriately chosen games, could be a good approach.

In fact, given the relevance of motivational aspects in the list, taking a better look at existing COTS programming games, even when their core design principles are based on making the game fun, makes sense. It could be suggested that researchers in the digital educational games field should start paying more attention to the game thinking design process and what makes a game engaging [37]. Therefore, focusing more on a multidimensional approach regarding the learners’ engagement, namely cognitive, behavioral, social, and affective [38]. This would be in contrast with traditional research on this topic, which tends to focus on the instructional content and evaluation of students based on the facts delivered in the digital games.

Consequently, the purpose of this study is twofold. First, to test whether the appropriate COTS programming games can be found and thus be used as effective educational tools, taking advantage of their engagement techniques, availability and affordability, compared to having to justify the cost of creating of a bespoke serious game from scratch, as remarked by [19]. And second, to study whether these games can serve to engage and help an adult audience, with its own idiosyncrasies that widely differ from younger students.

In order to study these possibilities, the following research questions in regard to the chosen COTS games (i.e. HRM and 7BH) are proposed:

RQ1: Are HRM and 7BH perceived as useful educational tools for CS1 courses?

RQ2: What was the user experience provided by HRM and 7BH? (i.e. are they easy to learn, self-contained, engaging games?)

3.2. Context of the study

The study was held during an academic semester at an online university offering official graduate and postgraduate programs fully recognized by the national government. About 75.000 students are enrolled each semester, most of them fully autonomous adult learners that cannot attend in face-to-face universities due to personal responsibilities, 64% of them being older than 30 years old (27% in the 40+ age category). According to the university’s internal statistics, the typical student works full-time and is financially independent (95%), is married (73%), and has children (58%). Hence, providing a student-centered educational model based on digital resources that rely on self-study, adaptable to their own time schedule, engaging, and perceived as immediately useful in their professional field is considered of utmost importance.

More specifically, the study was conducted in different instances of the Programming Fundamentals (ProgFun from now on) course. The ProgFun course is a single semester long mandatory subject during the first year of several graduate and postgraduate studies that rely on programming competences, mainly three separate bachelor studies: Computer Science, Telecommunication Engineering, and Data Science. In the case of postgraduate studies, this course must be passed before a student can join any program that requires knowing how to code.

In all cases, it is assumed that students have no previous technical background or knowledge in computer programming (even though it may not always be the actual case), apart from the basic expected digital competencies in a student enrolling in an online university (e.g. computer use, Internet skills).

The curriculum design, mainly based on the CS2013 with some adaptations from the CC2020, is divided according to three thematic blocks, each 6, 3, and 6 weeks long, respectively. In the first block, the basics of programming are studied, focusing on the most abstract concepts and basic language syntax. Pseudocode is used to represent algorithms at the very beginning, keeping the course language-agnostic when the approach is more conceptual, and real programming languages are introduced towards the end (a choice of C or Python, depending on the enrolled degree). The second block introduces concepts such as modularity, structs, pointers and input/output parameters. Finally, the last block is fully dedicated to more complex data types, search strategies, etc. There is an exception in the Data Science degree, where the last block is dedicated to Python libraries for data processing, analysis, and visualization. In any case, the subject is assessed on a weekly basis, with small mandatory assignments that must be submitted to the teacher, two half-semester long projects, and a final exam. Therefore, it is considered, by both students and teachers, as one of the most demanding courses in each degree.

3.3. Game selection process

Even though the chosen games have been presented as part of the background in Section 2.2.2, its choice was the result of following the methodology presented in this section, and not a decision taken *a priori*. As the first step, the vast amount of programming games with an educational aspect that can be found in the scientific literature, as presented in Section 2.3, was reviewed (researchers played at least 1 h). However, on further inspection, some of them were considered of a very limited scope and dissemination, purely experimental or currently abandoned. Additionally, it was found that some of them were not about programming at all, despite being labeled as such.

The next step was to identify the list of relevant games, able to specifically address the most challenging aspects in learning introductory computer programming following the conclusions in the review by [12] and the list of key programming topics identified in Section 2.1: engagement and problem-solving first, then student confidence and control structures. Nevertheless, apart from the educational content itself, which is indispensable, the game had to be actually engaging and fun, and not extremely difficult, with an adequate degree of usability and an approachable learning curve. Beyond the inherent qualities of the game during play, there were other aspects related to its ease of use that were also considered, so the game could be deployed in the broadest set of scenarios with minimum friction (e.g. installation process, compatible platforms, cost, etc.). All this additional aspects were also considered equally important, apart from the pure educational content.

Given these starting pointers and the context of the study, the researchers, all of them teachers of several programming subjects in graduate and postgraduate programs, agreed that the study would focus on the first block of the ProgFun course. Then, jointly proposed the following list of desirable properties, categorized by requirement levels, following the formal nomenclature in [39].

- It MUST focus on the more abstract and conceptual parts of the subject (problem-solving using algorithms) and control structures.
- It MUST be a COTS finished game (i.e. no Early Access or experimental), and still supported by the developers. Not an abandoned project.
- It MUST have good production values (graphics, interface, etc.), but low hardware requirements (e.g. usable in non-gaming laptops, or even tablets).

- It MUST have high user review ratings. Colloquially speaking, a fun and well-liked game.
- It SHOULD have a simple installation process, with no extra requirements (i.e. no mandatory on-line platform/services, or DRM).
- It SHOULD be easily available for different platforms, which the user can choose from.
- It SHOULD be able to allow the player to debug minimally the program. Step-by-step execution, at least.
- It MAY be language agnostic (i.e. not use a real programming language).

Given these requirements, it was not considered indispensable that the game used textual code, or even a real programming language, to guarantee the language-agnostic requirement, since different programming languages are used in the ProgFun course depending on the student profile (e.g. engineer or data scientist). Moreover, it was considered that using a simpler method of program representation (such as blocks) could let the game emphasize the problem-solving and more abstract aspects of computer programming, the actual focus of the study, instead of having to address the particularities of language syntax, such as compilation or interpreter errors. Eligible games were also required to stress flow control and value manipulation.

After the selection process, HRM and 7BH were considered the best candidates, although a few other games were close choices. Both games have excellent review scores, a 90.35% and 89.03% rating as of January 2024 respectively in the Steam digital store, or a 4.4 rating in the Android Play Store. In general, they are well-known among the fans of the genre, topping the lists of recommendations. Additionally, although not really necessary for this study, their age rating classification is PEGI 3, suitable for all age groups, although the complexity of the challenges would make them quite difficult for children.

Both games easily available for a wide breadth of platforms: PC, Linux, Mac OS X, Android, iOS, and even the Nintendo Switch and Wii U consoles. In the case of the computer version, they can be acquired for Windows, Linux and Mac from different digital distribution stores. However, on further examination of the available platforms, the preferred option would be to use the “Good Old Games” (GOG) store. This digital store provides a single DRM-free executable for installation with no other requirements. Neither other mandatory additional programs/launchers, nor a systematic authentication process once the games are downloaded and installed (unlike games acquired from Steam, for instance).

The CS2013 aligned learning outcomes and topics covered by HRM and 7BH, according to the literature (see Section 2.3), and on further analysis during the play sessions in this study, are:

1. Properties of Algorithms
2. Problem-solving strategies
3. Variables and primitive data types
4. Expressions and assignments
5. Simple I/O (conceptually, but not File I/O specifically)
6. Conditional and iterative control structures (using GOTO statements)
7. Debugging strategies
8. Documentation and program style

They basically encompass most of the topics in the “SDF/Algorithm Design” and “SDF/Fundamental Programming Concepts” in CS2013 (refer to Fig. 1), with a very strong focus on the abstract problem-solving part and control structures, which tend to be the most problematic ones for beginners, as previously stated. In regard to the ProgFun course itself, they would amount to the first thematic block approximately, during the 6 initial weeks of the course.

Even though the close choices were not finally included in the study, they are also worth mentioning, since they showed great potential

for further research work. They were *TIS-100*, *ExaPunks* and *Shenzen I/O* [40–42], a subset of the programming games by the developer Zachtronics,⁶ which specializes in this genre and is highly regarded by the public. These three games have excellent production values, very high user ratings –around 90–92%–, and all of them are available DRM-free through different distribution channels. However, these games were found very challenging, and its reliance on a made up low level assembly programming language, with an enforcement of strict memory limit constraints, was considered too complicated for an introductory programming course. Despite this, it would be worth studying their usefulness as educational tools in courses with more experienced students.

Finally, after the game review process, it was also found that few games actually managed to cover some of the more advanced topics from week 6 onwards, which are either much more syntax related or quite complex (such as data types, objects and classes, or file I/O operations). The studies presented in Section 2.3 provide a clear picture of which topics are covered by which games, but two of the most relevant ones covering some of the advanced topics are *Screeps* and *Adventure Land* [43,44], both MMORPG-like games, with a bit of a competitive nature, based on the use of JavaScript language. The previously mentioned *ExaPunks* is also considered one of the few games that covers topics very similar to, but not exactly, file I/O operation. *ElseHeartBreakO* [45], a highly narrative game, practically a graphical adventure, is also significant because of its focus on object-oriented programming principles to let the main character manipulate the game world.

3.4. Enrollment and participants

First, the sampling method, enrollment process and research protocol design were submitted and approved by the university Ethics Committee. Likewise, any recommendations suggested were followed during the whole research.

The enrollment started one week before the start of the semester through an open call to all students who took ProgFun for the first time regardless of the fully online academic program which they belonged to (i.e. BSc. and MSc. degrees). The call was disseminated via personal tutors, who redirected students that showed interest to an on-line form asking for a contact e-mail address and their preference of language during the study. The form also presented two check boxes where students stated that their participation was completely voluntary and accepted being contacted by the research team at any time. Furthermore, the participants were informed beforehand that they could leave the study at any time, i.e. without having to state any reason.

Although HRM and 7BH have Linux and Mac OS computer versions, the minimum system requirements needed to be part of the study were a PC compatible computer with at least a Windows 10 operating system and a broadband Internet connection (exactly the same as any other learner enrolled in the university). Use of tablets (Android/iOS) was also considered acceptable. A total of 51 students (25.5% female and 74.5% male) took part in the study. As shown in Fig. 4, they were adults between the ages of 18 and 63 (mean: 33.352; std: 9.688; median: 31), with 24 (47%) fell into the 26–35 age group. The distribution of the participants by the academic program was as follows: BSc. in Computer Engineering (27, 52.9%), BSc. in Data Science (14, 27.5%), MSc. in Data Science (7, 13.7%), BSc. in Telecommunications Engineering (2, 3.9%), and MSc. in Video games Development (1, 2%).

3.5. Research methods

The present research was conducted during the first block of the ProgFun course, lasting 6 weeks. The researchers were not teachers in these courses, always acting as external, impartial observers. Although communication between the researchers and the participants was intended to be kept to a minimum, students were provided with technical support for the games when necessary.

The study was mainly designed using a single group qualitative approach, based on reflective journals as the main instrument to collect the students' experiences and opinions. The use of guided journaling is considered an ideal tool to gather information based on people's perceptions, feelings and habits [46–48]. Additionally, journals can capture the students' change in perspectives as they progress in their learning experiences with a degree of detail and personalization that can be difficult to measure with other methods [49]. Furthermore, journals are also especially effective at creating and reinforcing a bond between the teacher and the student, providing an additional layer of engagement, and enhancing the learning process itself by fostering reflection and critical thinking [50,51].

At specific stages of the study, auxiliary questionnaires based on quantitative assessment were used to collect students' opinions on their perception of self-efficacy [52] and the user experience offered by the games (UEQ) [53,54].

Students always had at their disposal a tutorial on how to access and install the games, and a full explanation of the study protocol. Any documentation sent to participants was always written in their chosen language during the enrollment process, usually their native one. In that regard, although it was expected that they would also communicate or write their journal entries in the same language, they were free to choose any that the researchers would be able to understand.

3.6. Data collection instruments

A combination of several instruments were used throughout the study, in order to assess different aspects of the student experience: reflective journals, self-efficacy tests, and a user experience questionnaire. Next, the features of each and the corresponding collection process are detailed.

3.6.1. Reflective journals

The guided journaling procedure asked participants to write their entries on a weekly basis by proposing a set of self-reflection questions that indicated the topics to be discussed. The questions were constructed following the general recommendations in [55]. This method aimed to avoid blank canvas paralysis and to help students put their thoughts in order to create a cohesive discourse. However, they were absolutely free on how to handle their writings and present the topics to be discussed. It is worth noting that the first entry was a test, where participants only had to introduce themselves and give some information about their background and expectations on the ProgFun course. This allowed researchers to give some feedback on writing style and discuss any doubts that could arise at the very beginning, ensuring a basic quality for future entries.

When each week started, researchers sent a document with the questions for that entry, and asked participants to deliver it by the end of the week. Next, all journal entries were collected by the researchers, and briefly inspected for correctness (e.g. reading comprehension, that the proposed questions had been actually addressed, etc.).

Apart from the questions, no specific assignments were given for the week, and the study never changed the usual tasks expected from a student in the ProgFun course. Play was absolutely voluntary, and participants could interact with the games on their own volition. Only a brief recommendation on which levels to play was included with each document at the start of the week, so gameplay would follow the

⁶ <https://www.zachtronics.com>.

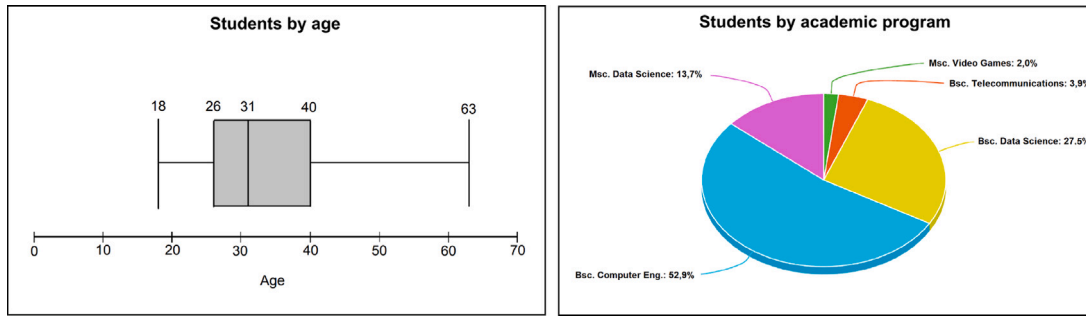


Fig. 4. Demographics of the participants.

Table 2
Opposite pairs of UEQ.

1.	Annoying	Enjoyable	14.	Unlikable	Pleasing
2.	Not understandable	Understandable	15.	Usual	Leading edge
3.	Dull	Creative	16.	Unpleasant	Pleasant
4.	Difficult to learn	Easy to learn	17.	Not secure	Secure
5.	Inferior	Valuable	18.	Demotivating	Motivating
6.	Boring	Exciting	19.	Does not meet expectations	Meets expectations
7.	Not interesting	Interesting	20.	Inefficient	Efficient
8.	Unpredictable	Predictable	21.	Confusing	Clear
9.	Slow	Fast	22.	Impractical	Practical
10.	Conventional	Inventive	23.	Cluttered	Organized
11.	Obstructive	Supportive	24.	Unattractive	Attractive
12.	Bad	Good	25.	Unfriendly	Friendly
13.	Complicated	Easy	26.	Conservative	Innovative

actual contents of the first block logically, but nothing was mandatory. Participants could take their time trying more advanced levels at their leisure, according to their spare time, interest, or how helpful they felt the games were. This was a crucial feature of the study, since evidence exists that what the literature refers as “mandatory fun” is actually counterproductive and could seriously impact the results [56,57].

Once the last expected journal entry was submitted by all the participants, a thematic analysis was performed on the collected journals, according to the guidelines on an inductive approach by [58]. Considering the objectives of the research and the key aspects identified in the journal entries, the researchers agreed on a coding frame composed of 7 categories: self-efficacy (with two additional subcategories: self-esteem and progress perception), engagement, user experience, satisfaction, stealth learning, curriculum alignment, and usefulness. Next, each researcher assigned text segments from the journals to the aforesaid categories and indicated for each one whether they had a positive or negative sense. After the individual analyses, researchers discussed the results and came to an agreement on the conclusions.

3.6.2. Self-efficacy tests

In order to further assess the changes in student expectations on their future performance at the very beginning of the course and after having freely interacted with the games during the study, the qualitative data from the journaling process was complemented with a self-efficacy test. Several tools exist to measure self-efficacy, but a simple single group pre-post test using Chen’s New General Self-Efficacy Scale [52] was presented to the students during weeks 1 and 6 of the study.

The New General Self-Efficacy Scale is an eight item test measure scale that assesses one’s belief that to achieve some expected goals, despite whatever difficulties may be encountered. Although this scale is considerably shorter than others, it is thought to have a higher construct validity. Using a five-point rating scale (from 1 = “strongly disagree” to 5 = “strongly agree”), respondents show how much they agree with each of the eight statements. A final score is calculated by averaging each rating, resulting in a range of values from 1 (low sense of self-efficacy) to 5 (high sense).

3.6.3. User experience questionnaire

The standard User Experience Questionnaire (UEQ) [59] was used to measure the students’ satisfaction with the user experience of the games. The questionnaire was a self-administered form with the 26 opposite pairs (see Table 2) required by the data analysis tool built by Schrepp et al. [53,54]. This was completed in weeks 2, when students had just briefly started interacting with the games, and 5, after they had ample opportunities to play with them. In both stages the participants were asked to fill out the form as quickly and spontaneously as possible, even if they were not sure whether a couple of terms clearly applied. For each pair, they had to choose a value between 1 and 7. Thus, depending on the chosen value, the user experience was considered closer to one of the two opposite items, e.g. slow (1) vs. fast (7).

After gathering all the responses, and following the indications defined by the UEQ data analysis tool, the values of each pair were transformed to a scale $[-3,+3]$, being the upper half of the values positive, the other half negative, and 0 the neutral value. However, according to Schrepp et al. it is extremely unlikely to observe values above +2 or below -2. Hence, those values between -0,8 and 0,8 represent a neutral evaluation, values greater than 0,8 are a positive assessment, and values lesser than -0,8 are negative. Finally, to facilitate data visualization, the UEQ data analysis tool transforms the 26 opposite pairs shown in Table 2 into 6 more general scales (see Table 3).

4. Results

This section shows the results of the study. They are presented from a qualitative and quantitative standpoint, depending on the related data collection methods and instruments, as described in Section 3.5.

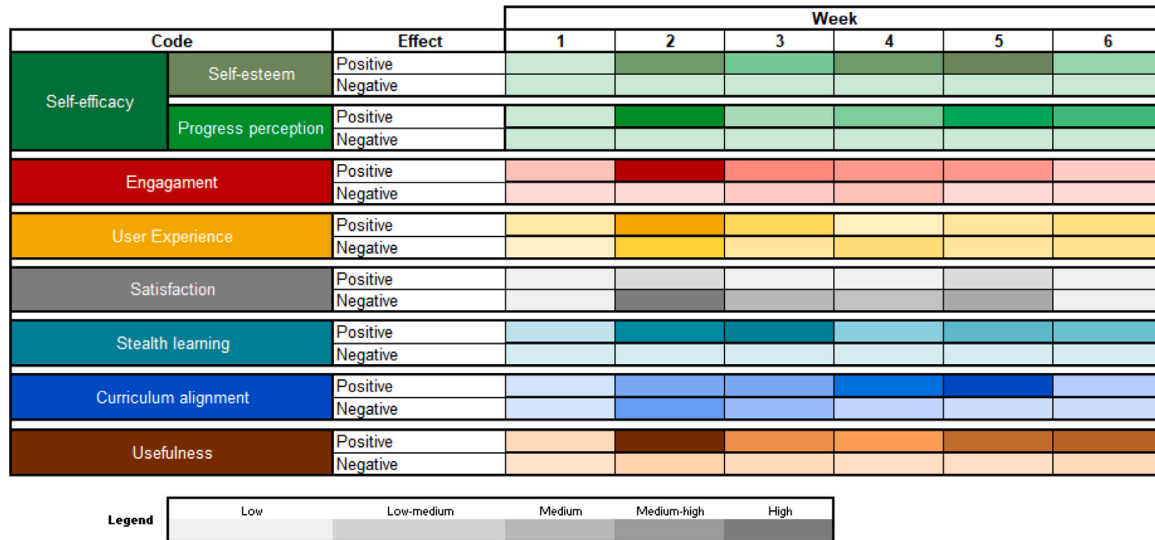
4.1. Thematic analysis from the journals

Fig. 5 summarizes the results of the thematic analysis. Each category is represented by a heatmap that displays its evolution according to the amount of text fragments identified as positive or negative throughout the six weeks of the study. The darkest the color is, the more text snippets were labeled.

Table 3

Definitions and pairs included in each UEQ scale.

Scale	Definition	Pairs included
Attractiveness	Overall impression of the games.	1, 12, 14, 16, 24, 25
Perspicuity	Ease of use of the games.	2, 4, 13, 21
Efficiency	Players can solve their tasks without unnecessary effort.	9, 20, 22, 23
Dependability	Players feel in control of the interaction.	8, 11, 17, 19
Stimulation	Players think the game is exciting and motivating to use.	5, 6, 7, 18
Novelty	The design of the games is creative	3, 10, 15, 26

**Fig. 5.** Heatmaps of the thematic analysis with the amount of text segments identified from the journals for each category of the coding frame throughout the experiment.

As for the progress in the sense of self-efficacy, assessed from a qualitative standpoint by means of the thematic analysis of the journal entries, one participant wrote in week 2: *“Not knowing programming requires doubling the necessary effort, but thanks to the two games, I’m noticing that I’m gaining an understanding that I didn’t have before, and I’ll be able to catch up with the classes soon”*. Also in week 2, another student described how HRM at level 17 increased his self-esteem: *“Finding the solution (which seems pretty easy once you have beaten the level) really inspired me and increased my confidence in my abilities”*. As the semester advanced and students played more, they felt more confident and perceived more progress, especially after week 4 (see Fig. 5): *“Every week I notice how I am gradually improving, not only in the course but also in the games, and it is comforting to feel how you are improving little by little, since you do not sometimes realize that you are progressing when studying”*.

In regard to engagement, and how the games succeeded in attracting and keeping students’ attention and interest and whether the games made players want to continue playing, the results show that the more weeks had passed since the beginning of the experiment, the lower the engagement.

User experience comprises both usability and some design issues that may affect the experience of students when playing. From a qualitative standpoint. As seen in Fig. 5, the second week was when more comments were written, either positive or negative. In fact, that week was when students really played the games for the first time. Learners stressed as positive the original, fun, and graphical way to apply programming concepts. Moreover, they appreciated that each level challenged them. However, they also expressed that they would have liked more hints or instructions on how to use some of the game programming commands.

Satisfaction means whether students felt pleased while using the games, instead of getting frustrated. In this case, several students got stuck and frustrated in some levels without knowing how to advance. This was especially disturbing when it occurred in week 2, since such

a situation caused some students to give up on the games. The same feeling happened when the levels started being perceived as more difficult, especially from week 5 onwards. In general, the design of the difficulty curve was one of the factors that more frustration caused, and hence, more negative comments. However, for some students, this actually became a feature, as exemplified by the excerpt: *“The game helped me (and a lot) to manage frustration when I got stuck at some level”*.

Some students wrote text fragments stating that they were learning programming concepts unconsciously while they were playing, i.e. stealth learning. Most of such snippets alluded to the benefit of playing to acquire computational and logical thinking skills, e.g., *“Game mechanics really helps me get into the mindset needed to program”* and *“this game helps me to think about the (graded) assignments in a way that I could not have done before playing it”*.

As observed in Fig. 5, students really realized the alignment of the games with the curriculum of the course towards the end of the journaling process (mainly, weeks 4 and 5). As evidence of this fact, one student in week 4 said: *“I think that until now, the game was ahead of the course. Now it seems that the course has reached the game and is going faster”*. In contrast, weeks 2 and 3 are the ones in which more comments pointed out that a mismatch was perceived. For instance, another student in week 2 commented: *“I think the game helps a lot to train the mind, however, for example, we are not working with loops yet. But the game helps me to train logical thinking”*.

Those text segments that mentioned the perception of usefulness of the games in relation to the course were assigned to this category. Results show that students wrote more positive ones than negative ones, especially during weeks 2, 5 and 6. Actually, in the journal entries of the last week, some students explicitly recommended using the games in future editions of the course, as supplementary learning material.

Table 4
Mean and confidence values for the 6 scales of the UEQ in weeks 2 and 5.

UEQ scales	Week 2 (N = 32)		Week 5 (N = 17)	
	Mean	Confidence	Mean	Confidence
Attractiveness	1,799	0,373	1,461	0,672
Perspicuity	0,586	0,289	0,221	0,371
Efficiency	1,531	0,330	1,181	0,654
Dependability	1,203	0,322	0,794	0,463
Stimulation	1,237	0,435	1,176	0,378
Novelty	1,659	0,343	1,515	0,686

4.2. Self-efficacy tests

The self-efficacy test was performed during the start of the course (week 1), and repeated at the very end of the study (week 6). The results from both tests, at the start (Mean = 4.02083; SD = 0.62348; SEM = 0.17998) and the end (Mean = 4.28125; SD = 0.56940; SEM = 0.16437) show that students had an elevated sense of self-confidence in their ability to follow the subject from the very beginning, already. Nevertheless, it further increased, and the data shows a two-tailed value of P equal to 0.0626, which could be considered statistically significant. These results back some of the previous topics and key statements detected in the journals about how the students' confidence on their ability to persevere and understand the topics of the subject progressed positively during the study because of the games.

4.3. Usability questionnaires

First, it is worth stressing that more students filled the questionnaire out in week 2 (N = 32) than in week 5 (N = 17). It can be hypothesized that its length discouraged the students, and some chose not to do it. Obviously this affects the results obtained in the second run. Nevertheless, all the means are positive both in week 2 and in week 5 (the range is [-3,+3]). The confidence interval with a significance level $\alpha = 0.05$ is between 0,289 and 0,435 in week 2, whereas it is between 0,378 and 0,686 in week 5. Such results imply that the means of the scores are relatively stable.

As shown in Table 4, the scales were ranked in the same order in both weeks. In this regard, attractiveness and novelty were the ones that obtained best scores, followed by efficiency, stimulation and dependability. In contrast, perspicuity was the worst rated scale despite having a positive mean.

5. Discussion

This section answers the questions posed in Section 3.1. In this regard, it is worth emphasizing that one of the most critical aspects of the games, that explains a lot of issues discussed in the present section, is the fact that they were not devised from an educational perspective. Nevertheless, the purpose of this study was precisely to assess their utility in spite of this, in order to take advantage of its production values, engagement, availability and no need for software development or maintenance.

5.1. RQ1 - are HRM and 7BH perceived as useful educational tools for CSI courses?

Generally speaking, according to the results and an in-depth analysis of student entries in the reflective journals, the games were well received and students considered them useful as an academic tool, even when the participants were adult learners with ages mostly around 26–40 years old, and, as adult learners, usually with other personal responsibilities and little spare time. The games were considered useful in the acquisition of computational thinking and problem-solving skills throughout all the study.

Even though some of the positive aspects in the thematic analysis slowly wore off as the novelty declined, the perception of usefulness was maintained consistently. Indeed, as early as weeks 2 and 3, some participants even stated that thanks to the games they were beginning to “think like a computer”, which made them realize their progress in learning how to solve programming problems. Also, a certain amount of them emphasized that the most useful levels were the ones that asked players to optimize the solutions, e.g. using the minimum steps to solve a problem. The more the participants played, the more they stated they felt they were progressing in the course. The results of the self-efficacy tests also show an increase in self-confidence, which can be linked mainly to the use of these video games, and not to other factors, given the comments in the reflective journals. Even though the degree of confidence was high from the very beginning, the journals allowed to detect a transition from simple optimism, at the start, towards actual realization of their potential to learn how to program.

A more specific aspect to take into consideration to fully answer this RQ is to assess their curriculum alignment and ability to actually let learners work on the expected concepts. This is relevant, since both HRM and 7BH are COTS games that were not developed with the ProgFun course in mind. In that regard, although the selection process of the games (see Section 3.3) took this into account from the educators' standpoint, learners did not always see a clear relationship between the course's curriculum –with a more formal presentation of theory–, and the games. As observed in Fig. 5, it took most of the participants until weeks 4 and 5 approximately, when they had learned all the theoretical concepts, to become aware of such alignment. This was further muddled a bit because of the games' approach to solve programming challenges.

Even though the purpose of choosing these games was purely to practice abstract thinking skills, and not coding syntax, the mismatch between the games' own code statements and actual coding caused some issues until the students had honed their abstraction skills. For instance, realizing that the *jump* statement used in the games was very similar to the use of the *for* statement of the programming languages. Nevertheless, the alignment existed, even though not necessarily 100%, since the learners kept realizing that, by playing the games, they had actually practiced many of the theoretical concepts (e.g. variables, control structures), showing also some degree of stealth learning in the process.

Therefore, even though the use of the games was considered positive by the students as a venue to hone their computational thinking skills, and had a positive effect in their perception of progress and self-esteem, results show the need to provide very clear guidelines on how the games must be consumed, instead of using a more free-form approach. For instance, it is necessary to develop a study guide on how each level relates to each lesson, and exactly when to play each level during the course. In fact, some students proposed grouping or labeling the game levels according to the concepts that were put in practice there. At the end, a game should not be treated differently from any other educational resource used in class.

5.2. RQ2 - what was the user experience provided by HRM and 7BH?

From the results, from both a qualitative and quantitative standpoint, participants liked both games. The use of both games scored positively in all categories in the UEQ, some higher than others, and also with some impact as the novelty wore off. More specifically, the user experience of the games was assessed positively in terms of aesthetics, with an appreciation to the novelty and originality of the game's metaphor when presenting its programming challenges (i.e. employees working at an office). From an analysis of some specific subitems in the UEQ, such as “Pleasing”, “Enjoyable” or “Motivating”, and several student comments in the qualitative assessment, it can be concluded that the games were actually fun and entertaining, and students had a good time interacting with them, one of the main goals in game based

learning. Again, this was regardless of the participants being adult learners. Indeed, students quickly realized that, under a look and feel that sometimes seemed childish, the challenges presented could quickly become really difficult, and up to the expected level of a CS1 course. Nevertheless, two very obvious problems were detected from a general user experience standpoint, in this educational context.

On one hand, and as shown by the “perspicuity” category in the UEQ, participants found it hard to get familiar with the game mechanics and some parts of the interface. Even though the initial levels are supposed to act as tutorials, and the technical background of some of the students, it cannot be assumed that any student is familiar with this kind of games. This was further exacerbated by the fact that the students were learning how to program and, therefore, how to create programs in the game was also not obvious at the very beginning. Given the results from the study, it becomes evident that the games expect players to already know some programming concepts in order to intuitively understand the interface and how to play, which is an aspect that must be taken into consideration when used at the very beginning of an introductory CS1 course.

On the other hand, frustration management was the biggest hurdle detected throughout the study, as proven also by the results of the negative “satisfaction” category in the thematic analysis. Even when students considered them engaging and useful for the course, and the act of playing the games was neither mandatory nor counted towards the final grade, being unable to complete a level would quickly demotivate them. There was a consensus that the increase in difficulty between levels was not smooth, with some sharp spikes, and many participants commented in their journals the need to have detailed solutions for each level, even when exchanging information among them in class was permitted.

In fact, this problem was detected even in more advanced students. Surprisingly, in their case, frustration was caused because they wanted to optimize and get a perfect score, even though they were able to solve the problem at hand. Also, towards the end of the study, one of the main reasons was that the games became repetitive once a high number of levels had been beaten. However, for some other students, all of this was really a feature, stating that trying to beat a level was a challenge that actually helped them to learn how to cope with frustration in real life.

As a result, it can be concluded that the games were considered fun and engaging, a mostly positive experience worth including in a CS1 course as an interactive educational resource that provides lots of feedback. In this case, their nature as COTS games geared towards entertainment played a positive role in the user experience, as hypothesized. However, and also because of this, it is clear that they cannot be used as self-contained educational resources. Again, the importance of a study guide that includes game instructions and a solution for each level, with a step-by-step explanation and some examples was made evident.

6. Conclusions and future work

At the start of this paper, it was hypothesized that COTS “programming games” could be helpful to students taking CS1 courses in reflecting on their algorithmic and computational thinking skills, despite being products designed first and foremost with entertainment in mind. This would allow educators to capitalize on their good production values, proven track record of engaging experiences, and convenient availability, obviating the need to develop (and maintain) bespoke games. According to this study, starting from the game selection process to the actual use of two well-known programming games in a real CS1 curriculum, this is an approach that shows potential, but only when some important points are taken into account. Especially, any candidate game must be subjected to the same degree of scrutiny as any other educational tool, and only a small subset of “programming games” are aligned to parts of the most popular CS1 curriculum definitions identified in the literature.

Since there are many games in this genre, the present study was based in only two of them, HRM and 7BH. Both are highly regarded among the fans of the genre, providing an interactive graphical interface based on code blocks, and some storytelling to contextualize the challenges presented to the player. Given the results, they seem to be attractive tools to learn computational thinking, even when presented to adult learners who are above the average age of the typical user of video games, and little free time because of personal responsibilities. In this regard, both seemed to present stimulating challenges that forced students to give better solutions to the problems until reaching optimized code. Moreover, thanks to the use of a fun context that proposed non-traditional exercises, students appreciated the fact that they learned without even being aware that they were being taught. The use of both games had a positive impact on their sense of self-efficacy and progress, especially in regard to problem-solving skills, and participants asked that both games are included in future editions of the course.

Nevertheless, due to the absence of an instructional design aligned to CS1 courses, it was made obvious that such games cannot be consumed without any guidance. Even if the task of playing is not made mandatory, students should have notes that not only indicate which levels to play, but also connect the levels to the concepts taught in the course so far, and provide tutorials and solutions. Or optionally, it is worth considering letting students freely share information among them to solve the challenges collaboratively, imitating what some successful gamer communities do (e.g. walkthroughs, “let’s play” videos).

A complementary approach would be to start using the games once all theoretical concepts have been learned, since it seems that “programming games” designers assume that players are already familiar with some programming concepts, and are not meant for someone who is starting from scratch. Even when this much more upfront approach loses some facets of “stealth learning”, it is advisable in order to avoid student frustration, either because they do not clearly see the alignment between the games and the curriculum, at the beginning, or because at some point they are unable to complete some of the challenges in the games.

As future work, and once the potential of both games has been established positively, the obvious next step would be to dedicate some effort to develop the pedagogical guides and test how this changes the full experience. Additionally, this study focused on abstraction skills, keeping motivation, and confidence, since they were identified among the top five difficulties according to the literature on teaching programming. However, there is ample room to perform different analyses on how other COTS programming games can be beneficial for other aspects, such as coding and language syntax. It would be interesting to study how more complex games, such as the close choices or the games including more advanced topics presented in Section 3.3, could help more experienced students to focus on better coding habits, while also making practice engaging and fun. In particular, games that rely on a real programming language syntax, such as *Screeps* or *Adventure Land*, both based on JavaScript.

CRedit authorship contribution statement

Joan Arnedo-Moreno: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **David García-Solórzano:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

References

- [1] F.J. García-Peñalvo, Editorial computational thinking, *IEEE Rev. Iberoam. Tecnol. Aprendiz.* 13 (1) (2018) 17–19.
- [2] S. Papert, An exploration in the space of mathematics educations, *IJ Comput. Math. Learn.* 1 (1) (1996) 95–123.
- [3] B. Trilling, C. Fadel, 21st Century Skills: Learning for Life in Our Times, John Wiley & Sons, 2009.
- [4] D. Loksá, A.J. Ko, W. Jernigan, A. Oleson, C.J. Mendez, M.M. Burnett, Programming, problem solving, and self-awareness: Effects of explicit guidance, in: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, 2016, pp. 1449–1461.
- [5] P. Tuomi, J. Multisilta, P. Saarikoski, J. Suominen, Coding skills as a success factor for a society, *Educ. Inf. Technol.* 23 (1) (2018) 419–434.
- [6] F.J. García-Peñalvo, A.M. Rees, J. Hughes, I. Jormanainen, T. Toivonen, J. Vermeersch, A survey of resources for introducing coding into schools, in: *Proceedings of the Fourth International Conference on Technological Ecosystems for Enhancing Multiculturality*, 2016, pp. 19–26.
- [7] M. Resnick, J. Maloney, A. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, A. Millner, E. Rosenbaum, J. Silver, B. Silverman, et al., *Scratch: programming for all*, *Commun. ACM* 52 (11) (2009) 60–67.
- [8] N.J. Coull, I.M. Duncan, Emergent requirements for supporting introductory programming, *Innov. Teach. Learn. Inf. Comput. Sci.* 10 (1) (2011) 78–85.
- [9] E. Lahtinen, K. Ala-Mutka, H.-M. Järvinen, A study of the difficulties of novice programmers, *ACM Sigcse Bull.* 37 (3) (2005) 14–18.
- [10] J. Bennedsen, M.E. Caspersen, M. Kölling, *Reflections on the Teaching of Programming: Methods and Implementations*, Vol. 4821, Springer, 2008.
- [11] C. Kazimoglu, M. Kiernan, L. Bacon, L. MacKinnon, Developing a game model for computational thinking and learning traditional programming through game-play, in: *E-Learn: World Conference on E-Learning in Corporate, Government, Healthcare, and Higher Education*, Association for the Advancement of Computing in Education (AACE), 2010, pp. 1378–1386.
- [12] R.P. Medeiros, G.L. Ramalho, T.P. Falcão, A systematic literature review on teaching and learning introductory programming in higher education, *IEEE Trans. Educ.* 62 (2) (2018) 77–90.
- [13] A. Luxton-Reilly, I. Albluwi, B.A. Becker, M. Giannakos, A.N. Kumar, L. Ott, J. Paterson, M.J. Scott, J. Sheard, C. Szabo, Introductory programming: a systematic literature review, in: *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*, 2018, pp. 55–106.
- [14] R.S. Dlamini, The role of the strategic and adaptive chief information officer in higher education, *Educ. Inf. Technol.* 20 (1) (2015) 113–140.
- [15] S.B. Khoza, Master of education students' reflections: which curriculum reasons are promoted or limited by skype resources? *Progressio* 39 (2) (2017) 1–19.
- [16] M. Prensky, Digital game-based learning, *Comput. Entertain. (CIE)* 1 (1) (2003) 21.
- [17] E.A. Boyle, T. Hainey, T.M. Connolly, G. Gray, J. Earp, M. Ott, T. Lim, M. Ninaus, C. Ribeiro, J. Pereira, An update to the systematic literature review of empirical evidence of the impacts and outcomes of computer games and serious games, *Comput. Educ.* 94 (2016) 178–192.
- [18] T.M. Connolly, E.A. Boyle, E. MacArthur, T. Hainey, J.M. Boyle, A systematic literature review of empirical evidence on computer games and serious games, *Comput. Educ.* 59 (2) (2012) 661–686.
- [19] J. Kirriemuir, A. McFarlane, Use of computer and video games in the classroom, in: *DiGRA Conference*, 2003, pp. 1–12.
- [20] J. Kirriemuir, A survey of COTS games used in education, in: *Serious Games Summit/Game Developers Conference*, 2005.
- [21] Tomorrow Corporation, Human resource machine, 2015, Game [Steam], Last played September 2022.
- [22] Tomorrow Corporation, 7 billion humans, 2018, Game [Steam], Last played February 2020.
- [23] A. Bandura, S. Wessels, Self-efficacy, 1994.
- [24] ACM/IEEE-CS Task Force on Computing Curricula, *Computer Science Curricula 2013*, Technical Report, ACM Press and IEEE Computer Society Press, 2013.
- [25] ACM/IEEE-CS Task Force on Computing Curricula, *Computing Curricula 2020*, Technical Report, ACM Press and IEEE Computer Society Press, 2020.
- [26] B.S. Bloom, et al., *Taxonomy of Educational Objectives. Vol. 1: Cognitive Domain*, Vol. 20, McKay, New York, 1956, p. 1, (24).
- [27] B.A. Becker, T. Fitzpatrick, What do CS1 syllabi reveal about our expectations of introductory programming students? in: *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, 2019, pp. 1011–1017.
- [28] G. Rodrigues, A.F. Monteiro, A.J. Osório, Introductory programming in higher education: A systematic literature review, in: A. Simões, J.A.C. Silva (Eds.), *Third International Computer Programming Education Conference, ICPEC 2022*, in: *Open Access Series in Informatics (OASICS)*, vol. 102, Schloss Dagstuhl–Leibniz-Zentrum für Informatik GmbH, Dagstuhl, Germany, 2022, pp. 4:1–4:17, <http://dx.doi.org/10.4230/OASICS.ICPEC.2022.4>.
- [29] M.J. Wolf, *The Medium of the Video Game*, University of Texas Press, 2001.
- [30] M.A. Miljanovic, J.S. Bradbury, A review of serious games for programming, in: *Joint International Conference on Serious Games*, Springer, 2018, pp. 204–216.
- [31] R. Hunnicke, M. LeBlanc, R. Zubek, MDA: A formal approach to game design and game research, *Proc. AAAI Workshop Chall. Game AI* 4 (1) (2004) 1722.
- [32] N. Lazzaro, Why we play: affect and the fun of games, in: *Human-Computer Interaction: Designing for Diverse Users and Domains*, Vol. 155, CRC Press Boca Raton, FL, 2009, pp. 679–700.
- [33] A. Vahldick, A.J. Mendes, M.J. Marcelino, A review of games designed to improve introductory computer programming competencies, in: *2014 IEEE Frontiers in Education Conference (FIE) Proceedings*, IEEE, 2014, pp. 1–7.
- [34] J. Arnedo-Moreno, D. García-Solórzano, Programming is fun! a survey of the steam digital distribution platform, in: *2020 IEEE 32nd Conference on Software Engineering Education and Training, CSE&T, IEEE*, 2020, pp. 1–4.
- [35] T.W. Malone, What Makes Things Fun to Learn? A Study of Intrinsically Motivating Computer Games (Ph.D. thesis), Stanford University, 1980.
- [36] C. Aldrich, *Learning by Doing: A Comprehensive Guide to Simulations*, Computer Games, and Pedagogy in E-Learning and Other Educational Experiences, John Wiley & Sons, 2005.
- [37] K. Xie, Projecting learner engagement in remote contexts using empathic design, *Educ. Technol. Res. Dev.* 69 (1) (2021) 81–85.
- [38] J.A. Fredricks, M. Filsecker, M.A. Lawson, Student engagement, context, and adjustment: Addressing definitional, measurement, and methodological issues, 2016.
- [39] S. Bradner, Key words for use in RFCs to indicate requirement levels, 1997, <http://dx.doi.org/10.17487/RFC2119>.
- [40] Zachtronics, TIS-100, 2015, Game [Steam], Last played February 2020.
- [41] Zachtronics, EXAPUNKS, 2018, Game [Steam], Last played February 2020.
- [42] Zachtronics, SHENZHEN I/O, 2016, Game [Steam], Last played February 2020.
- [43] K. Soral, Adventure land - the code MMORPG, 2019, Game [Steam], Last played February 2020.
- [44] Screeeps, Screeeps, 2016, Game [Steam], Last played February 2020.
- [45] E. Svedäng, Else heart.break(), 2015, Game [Steam], Last played February 2020.
- [46] J. Goetz, M. LeCompte, *Ethnological and qualitative design in educational research*, 1984.
- [47] V.J. Janesick, A journal about journal writing as a qualitative research technique: History, issues, and reflections, *Qual. Inq.* 5 (4) (1999) 505–524.
- [48] L. Cohen, Manion, L.-Morrison, K. Res. *Methods Educ.* 5 (2007).
- [49] D. Boud, Using journal writing to enhance reflective practice, *New Dir. Adult Contin. Educ.* 2001 (90) (2001) 9–18.
- [50] P.A. Connor-Greene, Making connections: Evaluating the effectiveness of journal writing in enhancing student learning, *Teach. Psychol.* 27 (1) (2000) 44–46.
- [51] P. Hettich, Journal writing: Old fare or nouvelle cuisine? *Teach. Psychol.* 17 (1) (1990) 36–39.
- [52] G. Chen, S.M. Gully, D. Eden, Validation of a new general self-efficacy scale, *Organ. Res. Methods* 4 (1) (2001) 62–83.
- [53] M. Schrepp, A. Hinderks, J. Thomaschewski, Construction of a benchmark for the user experience questionnaire (UEQ), *Int. J. Interact. Multim. Artif. Intell.* 4 (4) (2017) 40–44.
- [54] M. Schrepp, A. Hinderks, J. Thomaschewski, Design and evaluation of a short version of the user experience questionnaire (UEQ-S), *Int. J. Interact. Multim. Artif. Intell.* 4 (6) (2017) 103–108.
- [55] J.C. Dunlap, Using guided reflective journaling activities to capture students' changing perceptions, *Tech Trends* 50 (6) (2006) 20–26.
- [56] E.R. Mollick, N. Rothbard, *Mandatory Fun: Consent, Gamification and the Impact of Games at Work*, The Wharton School Research Paper Series, 2014.
- [57] G. Rodríguez-Aflecht, M. Hannula-Sormunen, J. McMullen, T. Jaakkola, E. Lehtinen, Voluntary vs compulsory playing contexts: Motivational, cognitive, and game experience effects, *Simul. Gaming* 48 (1) (2017) 36–55.
- [58] D.R. Thomas, A general inductive approach for analyzing qualitative evaluation data, *Am. J. Eval.* 27 (2) (2006) 237–246, <http://dx.doi.org/10.1177/1098214005283748>.
- [59] B. Laugwitz, T. Held, M. Schrepp, Construction and evaluation of a user experience questionnaire, in: A. Holzinger (Ed.), *HCI and Usability for Education and Work*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 63–76.